■ System Design: YouTube Recommendation System

1. Problem Definition & Requirements

Objective:

Deliver real-time, highly relevant, and personalized video recommendations to billions of global YouTube users.

Key Use Cases:

Personalized home feed and "Up Next" queue.

Dynamic ranking for watch, search, and trending pages.

Email and push notification video suggestions.

Cross-device/session consistency.

Requirements:

Latency: Sub-100ms recommendation response time.

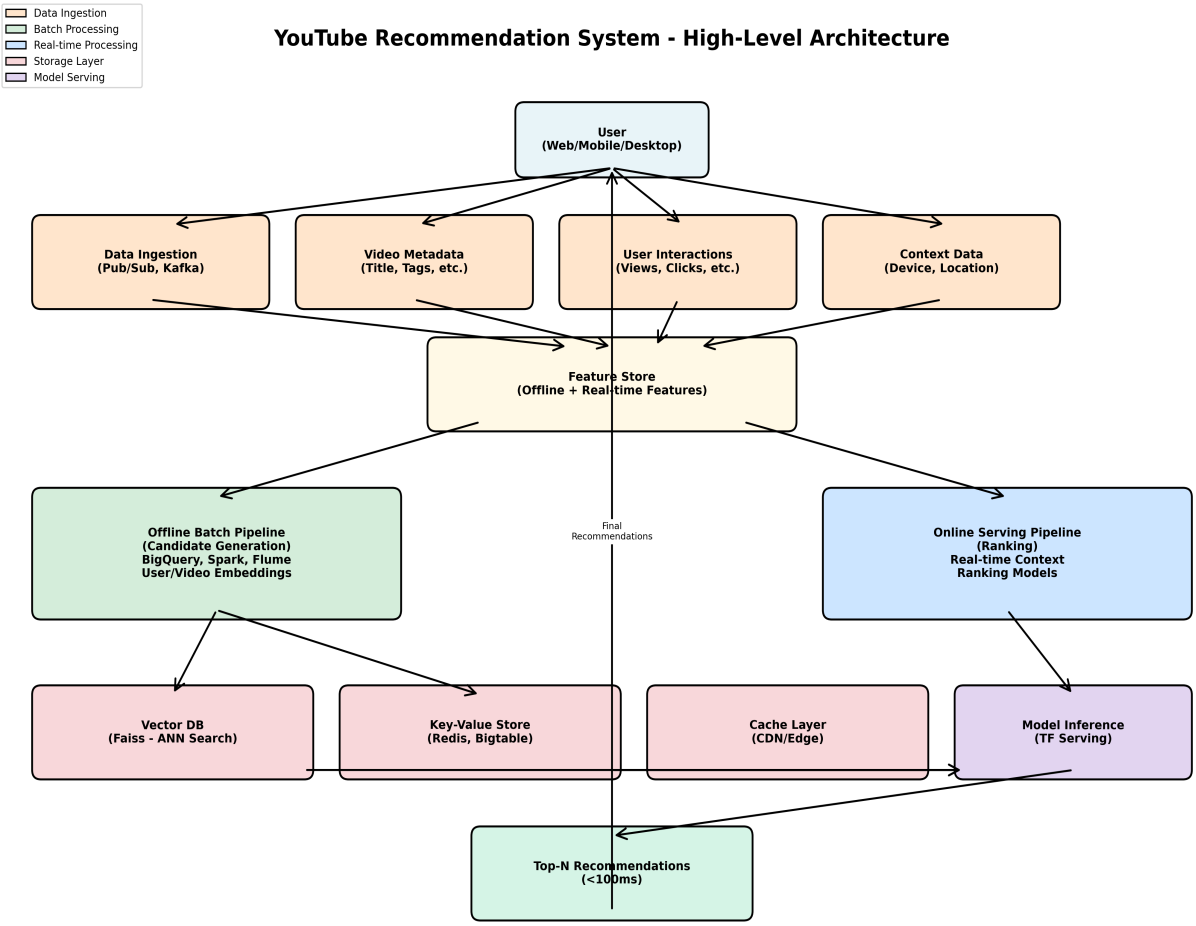
Scalability: Billions of recommendations per day, millions of videos/users.

Adaptability: Fast reaction to user trends, trending topics, and breaking news.

Diversity: Avoid echo chambers; promote variety and serendipity.

Fairness & Compliance: Adherence to regional guidelines, copyright, and user safety.

2. High-Level Architecture



Core System Components

Data Ingestion:

Streams of user interactions (views, watches, likes, comments, shares, subscriptions) and video metadata collected using distributed log systems (e.g., Pub/Sub, Kafka).

Feature Store:

Centralized store for both historical (offline) and session-based (real-time) features—supports context, user, and content features required during training and inference.

Offline Batch Pipeline (Candidate Generation):

Large-scale processing (e.g., with BigQuery, Spark, Flume) to periodically generate user and video embeddings based on long-term behavior.

Online Serving Pipeline (Ranking):

Consumes live session data, fetches candidates, and applies advanced ranking models to create up-to-the-moment recommendations.

Model Inference API:

Rankings computed via deep learning model APIs (TensorFlow Serving, custom C++/Go microservices), supporting real-time request volume.

Storage:

Key-value stores (Redis, Bigtable) for fast access to session data, vector DBs (like Faiss) for ANN-based candidate lookups.

Supporting Key Use Cases

Email and Push Notification Video Suggestions:

The system supports email and push notifications through:

Offline Batch Pipeline: The candidate generation system that runs periodically (using BigQuery, Spark, Flume) can be used to generate recommendations for batch processes like emails.

Feature Store: Historical user preferences and engagement profiles can feed a separate batch job for email/push recommendations.

Model Inference API: Can be called asynchronously for notification generation.

Additional Components: Notification scheduling service, email template generation, push notification delivery infrastructure, and optimal timing algorithms for sending notifications.

Cross-Device/Session Consistency:

The system ensures consistency across devices and sessions through:

Feature Store: Centralized store for both historical (offline) and session-based (real-time) features maintains user state across devices.

User Embeddings: Generated in the offline batch pipeline based on long-term behavior at the user-level (not device-level).

Data Ingestion: Collects user interactions tied to user accounts rather than specific devices.

Session Synchronization: Mechanisms for session state storage, retrieval, and propagation of recent session data across devices through user authentication/identification.

3. Data Pipeline Design

Data Sources:

Real-time user signals: Watch time, video completion, skips, rewinds, subscriptions, likes/dislikes, search queries.

Video/content metadata: Title, tags, categories, upload time, description, thumbnails.

Contextual signals: Device, location, language, time of day, user session flow.

Explicit feedback: Explicit ratings and feedback from user controls.

ETL and Feature Engineering:

Batch Layer: Aggregates historical user-video interactions, creates long-term preference and engagement profiles.

Streaming Layer: Captures and updates session-based behaviors for recent or new users.

Feature Store: Ensures feature consistency and availability for both model training and live recommendations.

4. Model Design & Training

Stage 1: Candidate Generation (Batch Layer)

Goal: Narrow down from hundreds of millions of videos to a pool of ~10,000 relevant videos per user in near real time.

Techniques:

Collaborative Filtering: User-to-video and video-to-user matching based on implicit feedback (watch time, engagement).

Content-Based Filtering: Leverages tags, text description, video embeddings, language, and category similarity.

Session-Contextual Filtering: Uses trending topics, regional trends, freshness, and history.

Popularity/Trend Signals: Factors in virality, recent surges, and external upticks.

Implementation:

Embeddings for users/items precomputed periodically; vector similarity search (e.g., Faiss) applied for nearest neighbors; candidate sets cached in distributed key-value stores.

Stage 2: Ranking (Real-Time)

Goal: From thousands of candidates, select the top-N (usually 10–20 for each surface) in real time.

Feature Inputs:

User: Watch history, interests, subscriptions, time since last interaction, device.

Video: Recency, past popularity, topic clusters, duration, creator data.

Context: Session signals, time of day, location, platform.

Cross features: User-video pair features, such as last interaction, or co-watch statistics.

Model Choices:

Deep Neural Networks: YouTube's typical approach is a two-tower neural network (one for user, one for video) for embedding creation, followed by a deep cross network for interaction modeling.

Sequence Models: Use RNNs or Transformers to incorporate sequential, temporal patterns in user behavior.

Loss Functions:

Pairwise ranking (e.g., BPR), multi-class SoftMax, or listwise ranking objectives focused on watch time, click-through rate, or dwell time.

5. Online Serving & Inference

Request Flow:

Step 1: User opens YouTube home or video/watch page.

Step 2: Session features (recent behaviors, device type) and user embeddings are fetched from storage.

Step 3: Approximate nearest neighbor search (ANN) (Faiss/Bigtable) retrieves top ~10,000 candidates.

Step 4: Ranking model reevaluates top candidates with up-to-the-second context.

Step 5: Final top-N personalized recommendations are returned within strict latency SLAs (<100ms).

Tools:

Faiss, Redis, Bigtable: For fast retrieval of embedding and candidate sets.

Model Serving: High-throughput inference via TensorFlow Serving, C++/Go microservices, or custom internal platforms for DNNs.

Caching: For anonymous or guest users, surface trending or regionally popular videos using CDN/front-edge cache.

6. Scalability & Deployment

Horizontal autoscaling for candidate generation, ranking, and inference services.

Auto-scaling groups and regional traffic balancing for global reliability.

Regular retraining and rollout of models with validation pipelines, shadow/canary deployments, and online A/B test cells to minimize regression risks.

7. Monitoring, Feedback, and Feedback Loops

System Monitoring: Track latency, throughput, error rates, and capacity metrics.

Recommendation KPIs: Watch time, CTR, user retention, bounce rate, diversity metrics (novelty, repetition).

Feedback Logging: All impression, click, and engagement events are logged, versioned by model/timestamp.

Feedback Loops: Use logged data for continual model refinement, hard-negative mining, and drift adaptation.

A/B Testing: Continuous experimentation with new algorithms or feature sets; use bandit strategies to explore/exploit recommendation policies dynamically.

8. Cold Start & Diversity Handling

New Users: Rely on context (location, device, time), minimal bootstrapping from trending content, demographics, or explicit preferences (onboarding).

New Videos: Use content embeddings and metadata to place new items into the candidate pool rapidly.

Diversity & Exploration: Apply diversity-promoting ranking methods (e.g., Maximal Marginal Relevance), editorial controls, and explicit novelty-promotion modules to balance personalization and discovery.

9. Privacy, Safety, and Explainability

Data Encryption: At rest and in transit for all personal and behavioral data.

Safety/Moderation: Filter recommendations for sensitive or policy-violating content, comply with child safety and regional restrictions.

Access Control: Role-based authorization for all sensitive user/model data.

Compliance: GDPR/CCPA support, with opt-out and user data export mechanisms.

Explainability: Use interpretable model overlays (LIME, SHAP, or internal visualization tools) where possible and needed for oversight and user trust.